



# 算力炼金术：昇腾950利用率提升的奥秘

演讲人 沈敏





# 大模型算子昇腾950极致优化

MXFP4 Quant Matmul

↑ 57%

RmsNorm

↑ 99%

Scalar

↑ 45%

充分利用Ascend950硬件特性，对算子性能极致优化，以完全释放950算力

了解更多

<https://gitcode.com/cann/cann-samples>

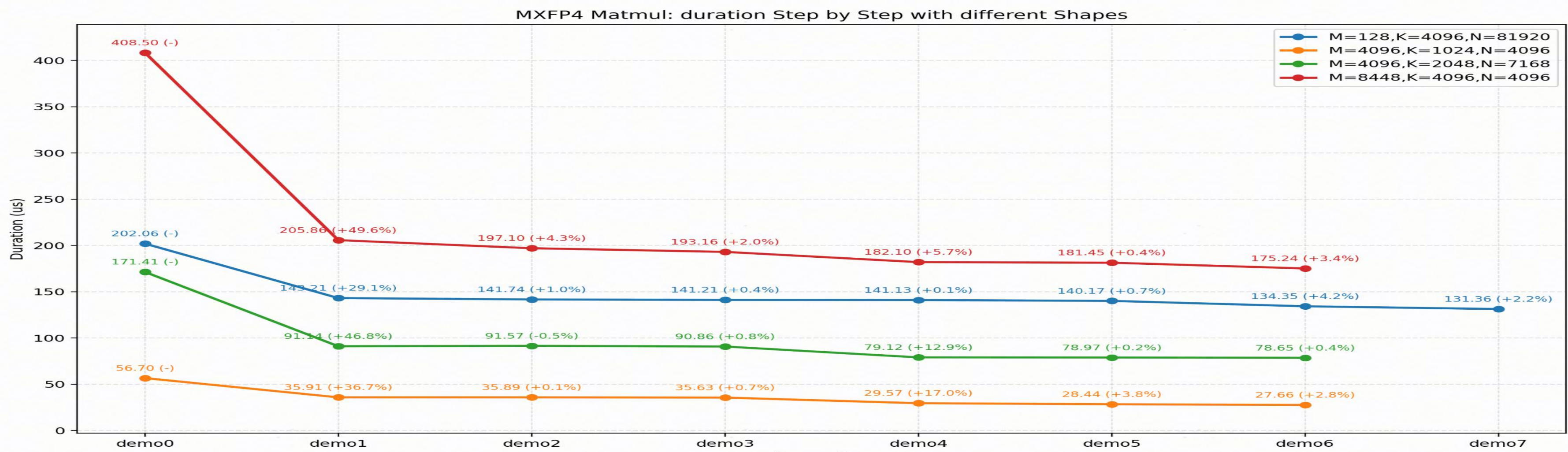


# MXFP4 Quant Matmul性能优化

57%

× 总加速比

408 us → 175 us



0

Naive 基线

Duration

408 us

累计加速

1.0×

1

Double buffer

Duration

205 us

累计加速

1.99×

2

SWAT调度

Duration

197 us

累计加速

2.07×

3

尾轮负载均衡

Duration

193 us

累计加速

2.11×

4

UnitFlag

Duration

182 us

累计加速

2.24×

5

L1 Bank Conflict优化

Duration

181 us

累计加速

2.25×

6

Scale缓存

Duration

175 us

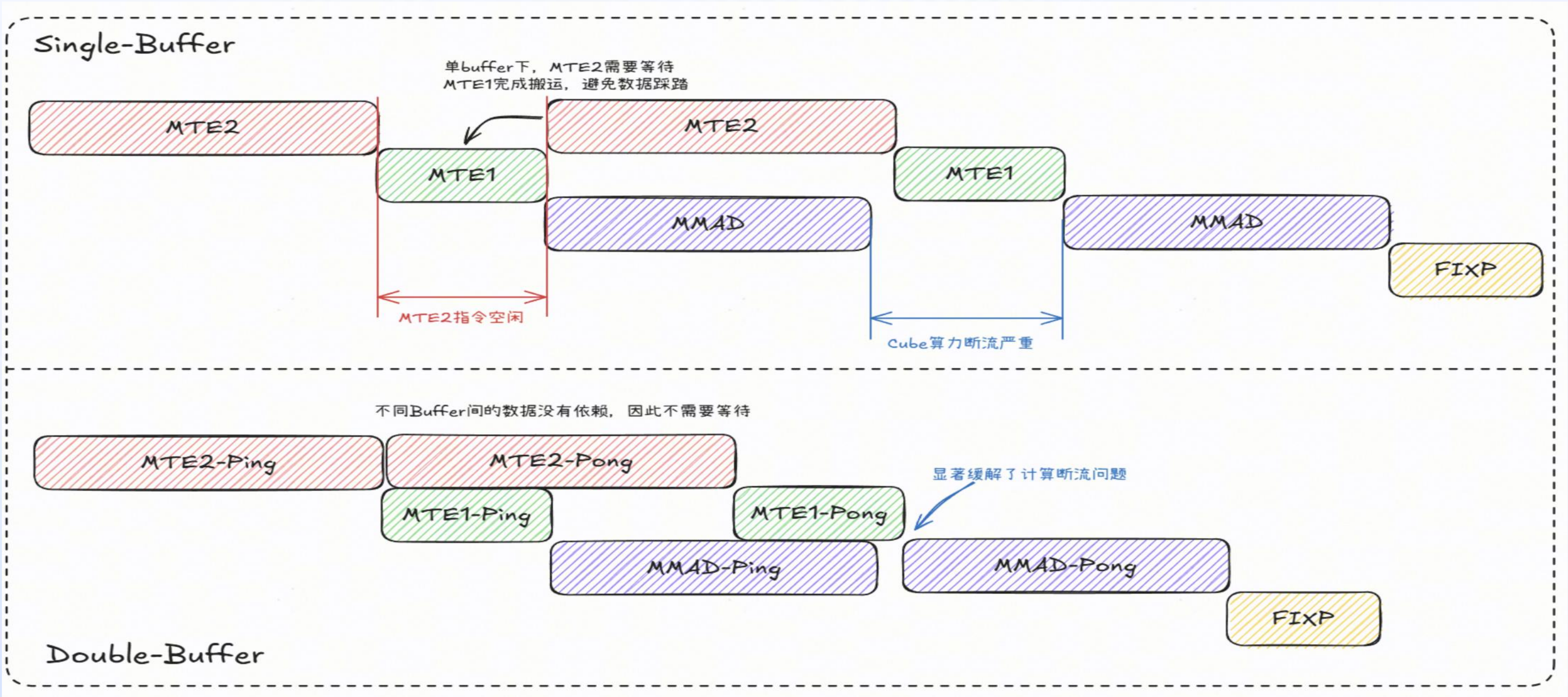
累计加速

2.33×



# MXFP4 Quant Matmul性能优化

|                                    |                                     |                                     |                                     |                                     |                                     |                                     |
|------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| 0                                  | 1                                   | 2                                   | 3                                   | 4                                   | 5                                   | 6                                   |
| Naive 基线                           | Double buffer                       | SWAT调度                              | 尾轮负载均衡                              | UnitFlag                            | L1 Bank Conflict优化                  | Scale缓存                             |
| Duration<br>408 us<br>累计加速<br>1.0× | Duration<br>205 us<br>累计加速<br>1.99× | Duration<br>197 us<br>累计加速<br>2.07× | Duration<br>193 us<br>累计加速<br>2.11× | Duration<br>182 us<br>累计加速<br>2.24× | Duration<br>181 us<br>累计加速<br>2.25× | Duration<br>175 us<br>累计加速<br>2.33× |





# MXFP4 Quant Matmul性能优化

0

Naive 基线

Duration

**408 us**

累计加速

**1.0×**

1

Double buffer

Duration

**205 us**

累计加速

**1.99×**

2

SWAT调度

Duration

**197 us**

累计加速

**2.07×**

3

尾轮负载均衡

Duration

**193 us**

累计加速

**2.11×**

4

UnitFlag

Duration

**182 us**

累计加速

**2.24×**

5

L1 Bank Conflict优化

Duration

**181 us**

累计加速

**2.25×**

6

Scale缓存

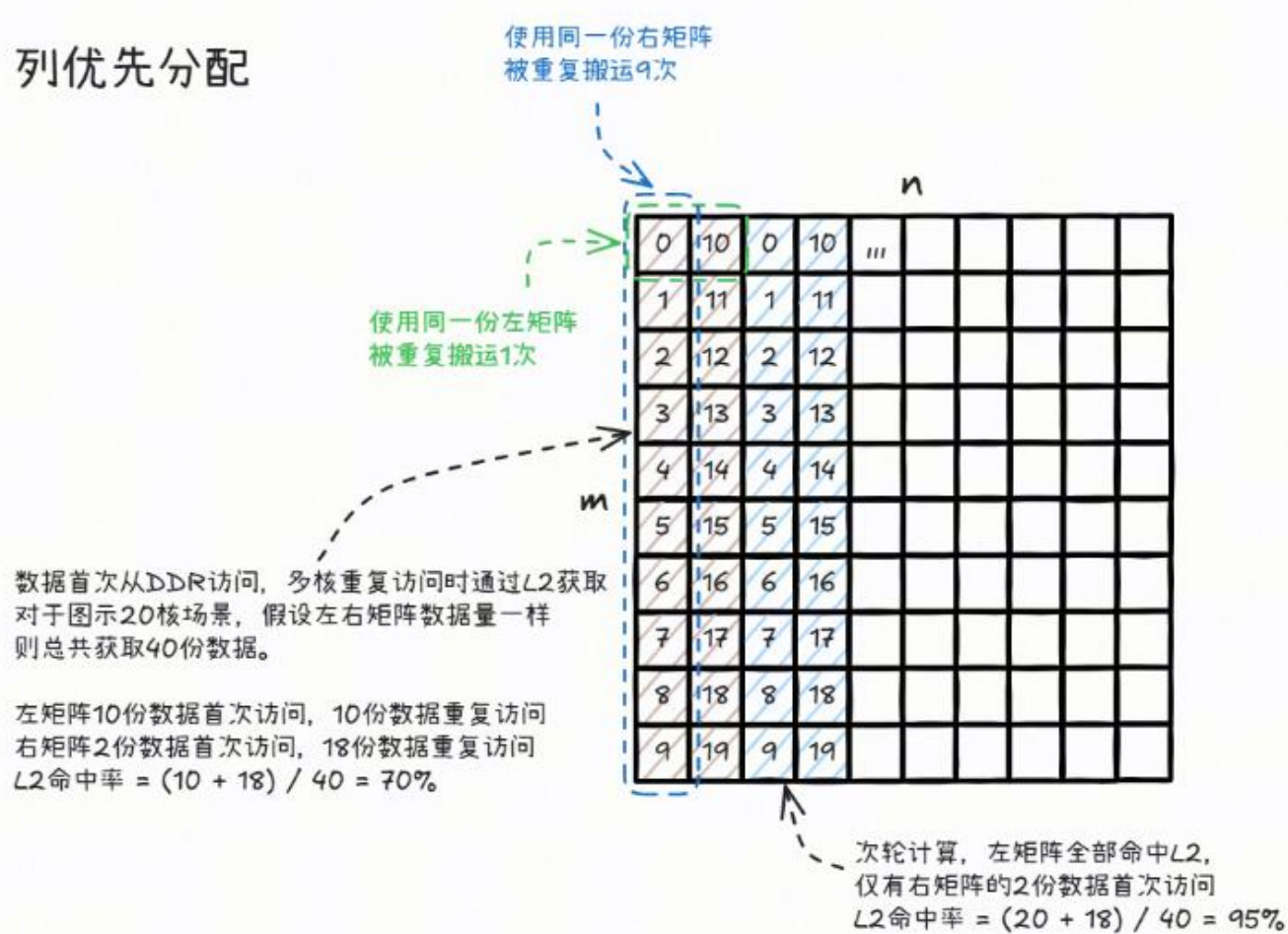
Duration

**175 us**

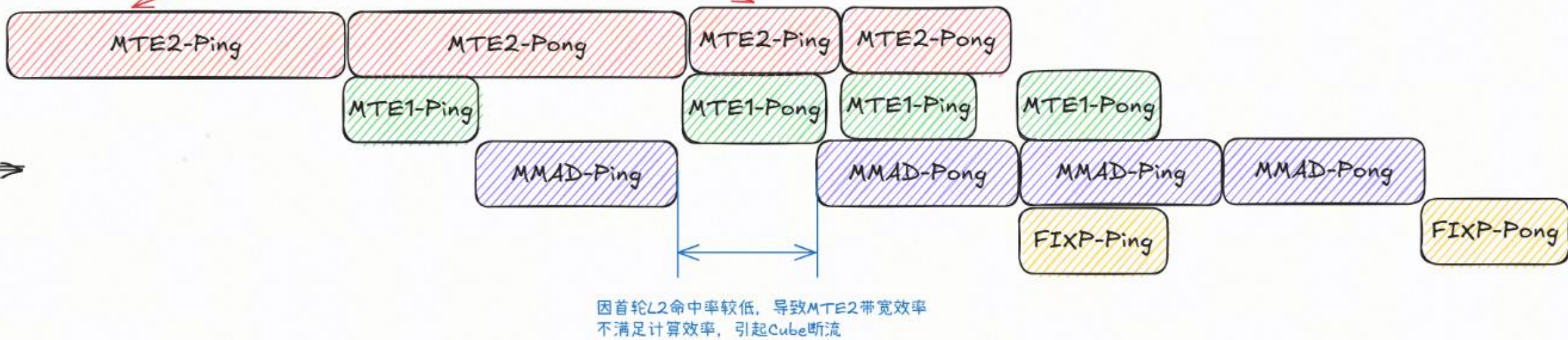
累计加速

**2.33×**

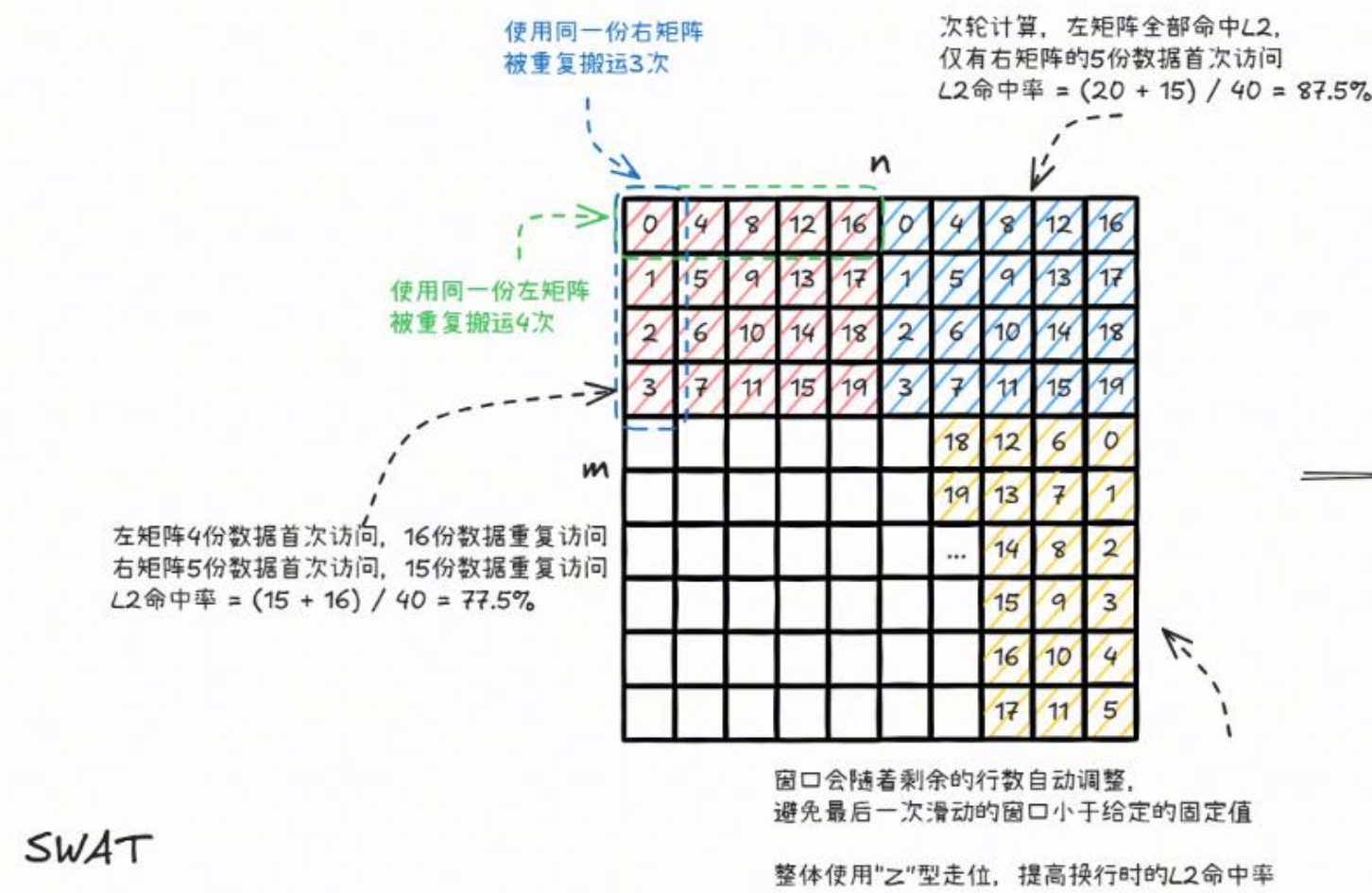
列优先分配



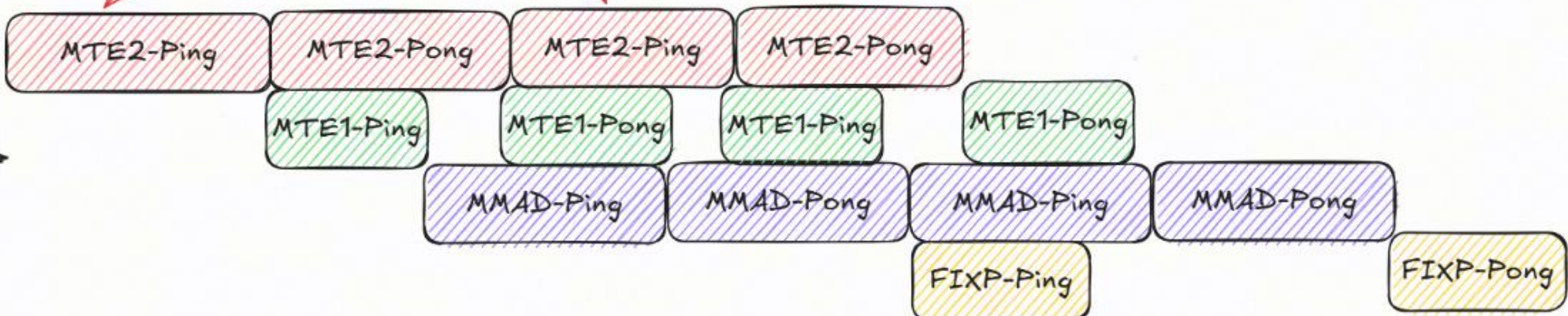
首轮L2命中率过低，导致MMAD断流  
次轮的L2命中率过高，导致访问带宽的浪费



因首轮L2命中率较低，导致MTE2带宽效率  
不满足计算效率，引起Cube断流



首轮和次轮的L2命中率更均匀，整体的使得首轮执行即可保证MMAD连续执行





# MXFP4 Quant Matmul性能优化

0

Naive 基线

Duration

408 us

累计加速

1.0×

1

Double buffer

Duration

205 us

累计加速

1.99×

2

SWAT调度

Duration

197 us

累计加速

2.07×

3

尾轮负载均衡

Duration

193 us

累计加速

2.11×

4

UnitFlag

Duration

182 us

累计加速

2.24×

5

L1 Bank Conflict优化

Duration

181 us

累计加速

2.25×

6

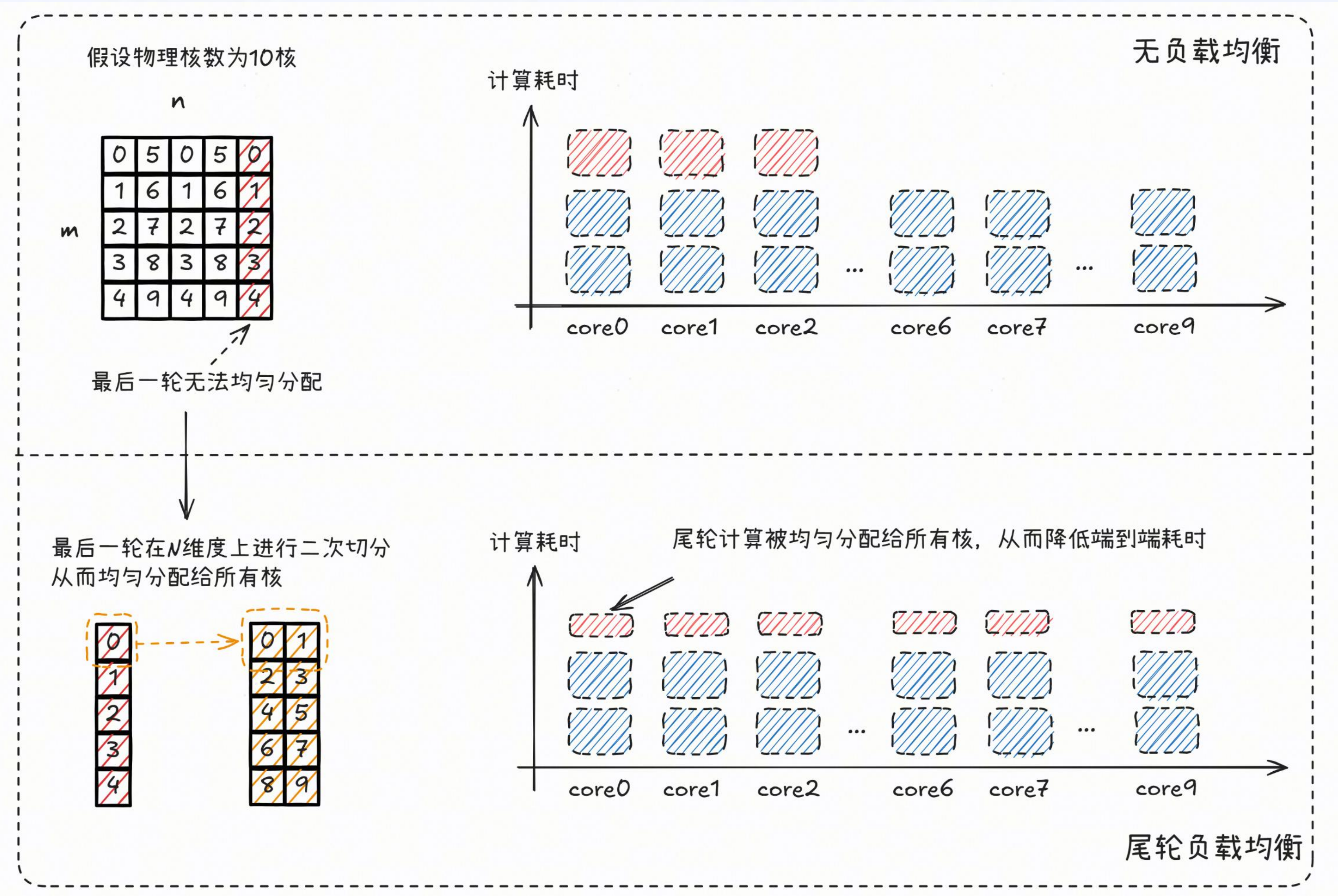
Scale缓存

Duration

175 us

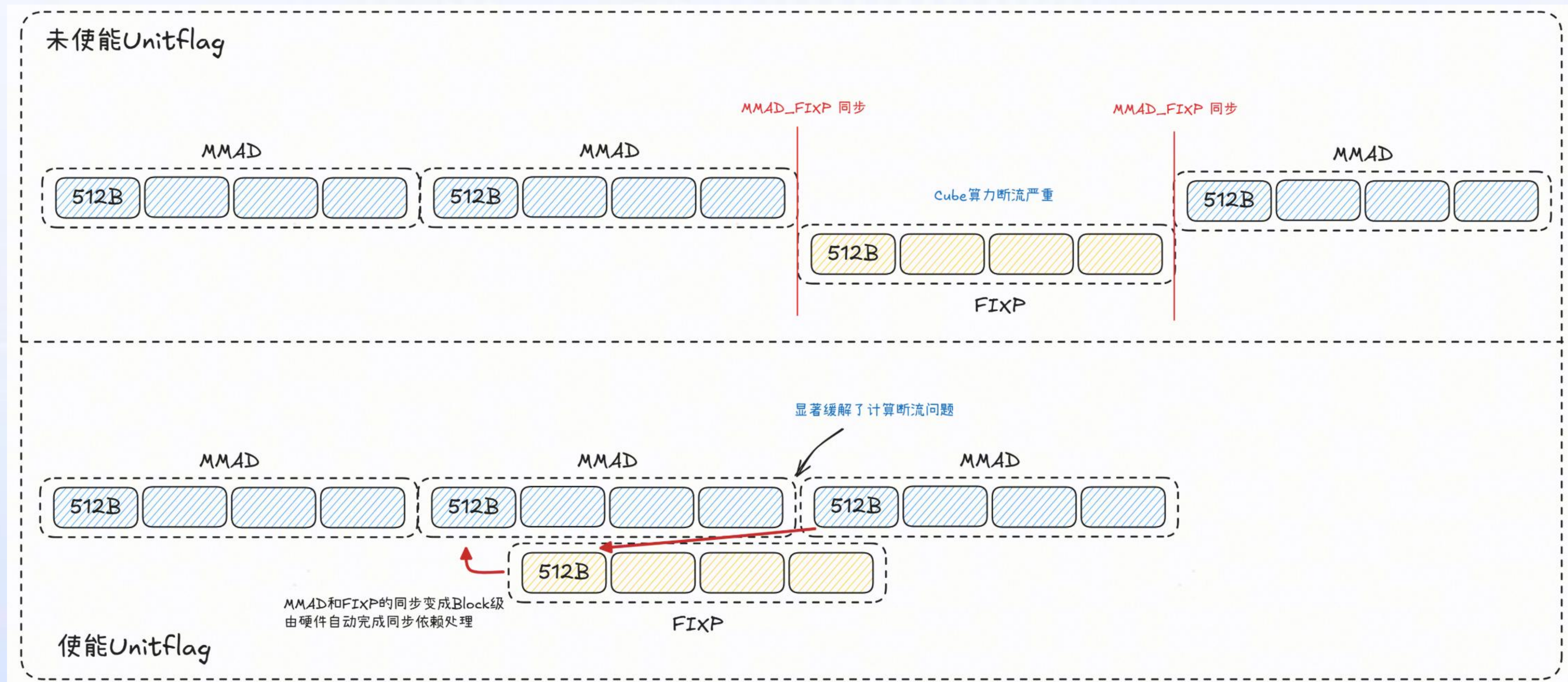
累计加速

2.33×



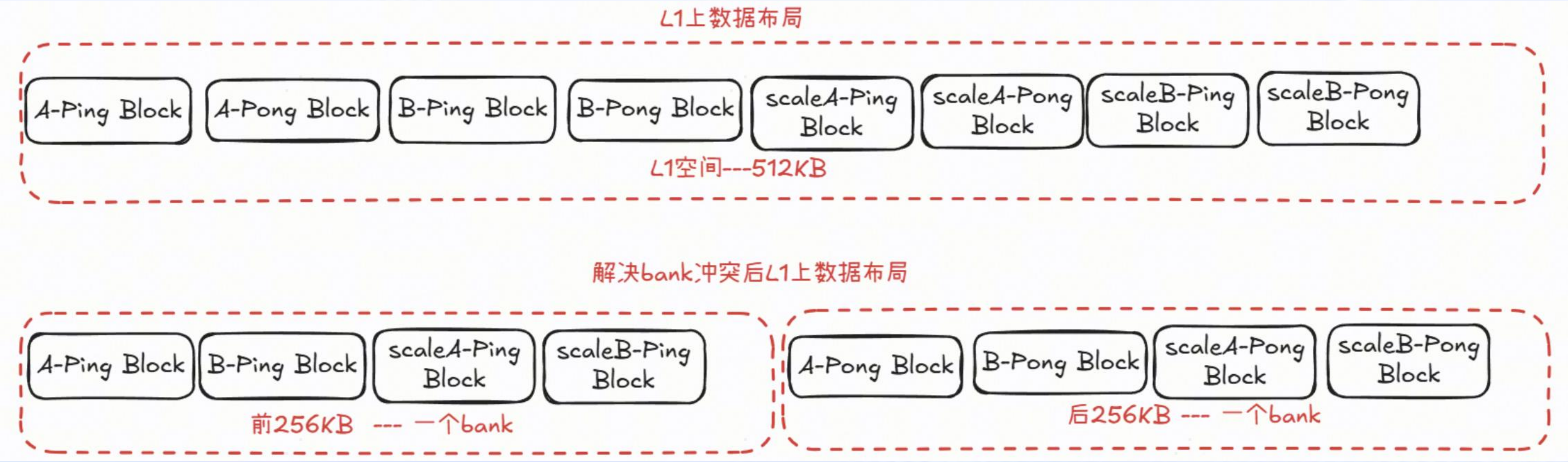


# MXFP4 Quant Matmul性能优化





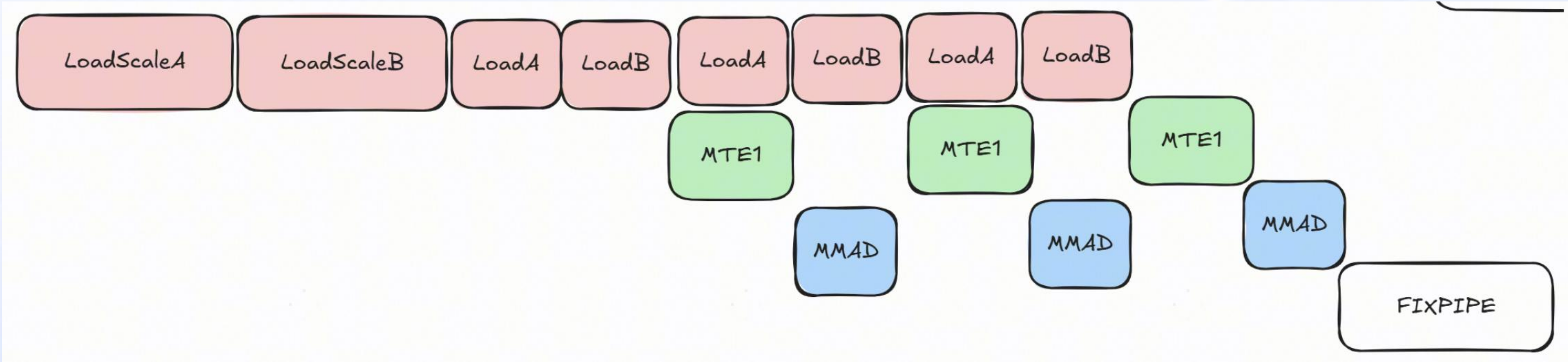
# MXFP4 Quant Matmul性能优化





# MXFP4 Quant Matmul性能优化

|                                    |                                     |                                     |                                     |                                     |                                     |                                     |
|------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| 0                                  | 1                                   | 2                                   | 3                                   | 4                                   | 5                                   | 6                                   |
| Naive 基线                           | Double buffer                       | SWAT调度                              | 尾轮负载均衡                              | UnitFlag                            | L1 Bank Conflict优化                  | Scale缓存                             |
| Duration<br>408 us<br>累计加速<br>1.0× | Duration<br>205 us<br>累计加速<br>1.99× | Duration<br>197 us<br>累计加速<br>2.07× | Duration<br>193 us<br>累计加速<br>2.11× | Duration<br>182 us<br>累计加速<br>2.24× | Duration<br>181 us<br>累计加速<br>2.25× | Duration<br>175 us<br>累计加速<br>2.33× |



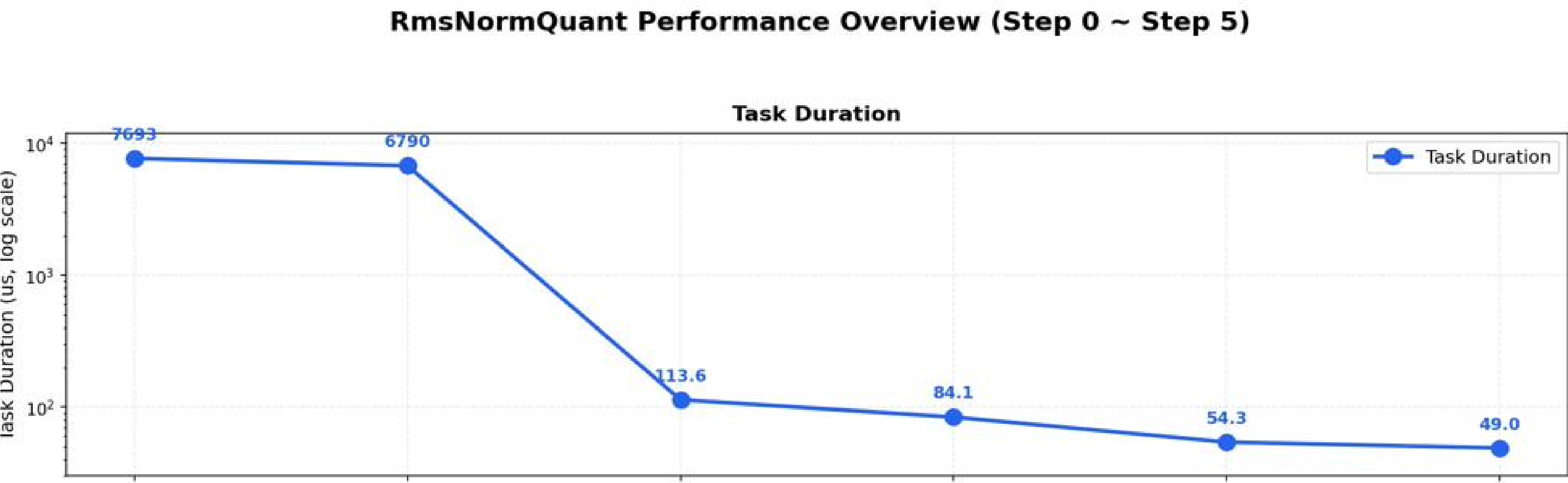


# RmsNorm性能优化

99%

× 总加速比

7693 us → 49.0 us



0

Naive 基线

Duration

7693 us

累计加速

1.0×

1

Gamma 预加载

Duration

6790 us

累计加速

1.13×

2

多核并行

Duration

113.6 us

累计加速

67.7×

3

RegBase重构

Duration

84.1 us

累计加速

91.5×

4

Double Buffer

Duration

54.3 us

累计加速

141.7×

5

UB 多行批处理

Duration

49.0 us

累计加速

157.0×

6

二分累加 + rsqrt

Duration

49.5 us

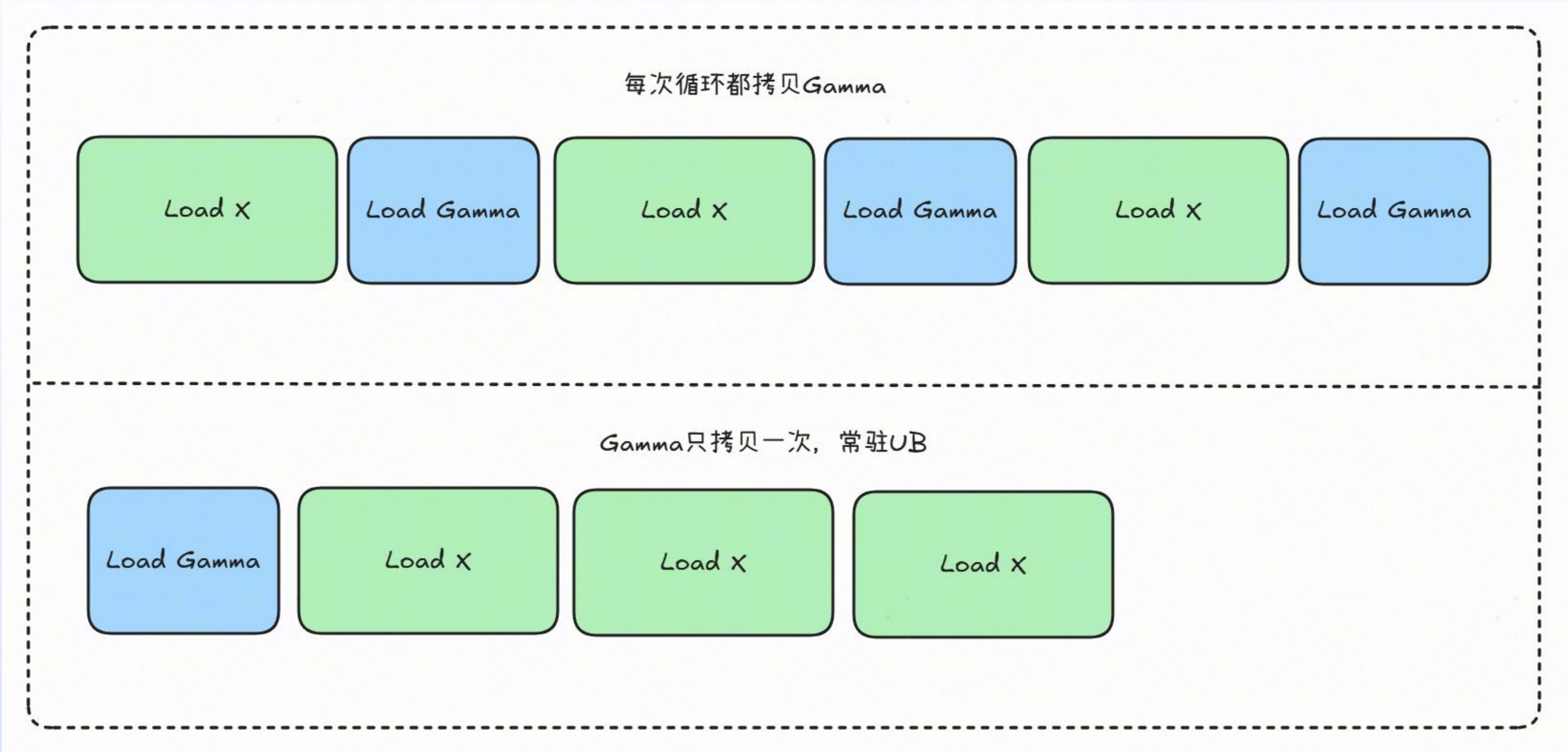
累计加速

—



# RmsNorm性能优化

|                     |                     |                      |                     |                     |                     |                     |
|---------------------|---------------------|----------------------|---------------------|---------------------|---------------------|---------------------|
| 0                   | 1                   | 2                    | 3                   | 4                   | 5                   | 6                   |
| Naive 基线            | Gamma 预加载           | 多核并行                 | RegBase重构           | Double Buffer       | UB 多行批处理            | 二分累加 + rsqrt        |
| Duration<br>7693 us | Duration<br>6790 us | Duration<br>113.6 us | Duration<br>84.1 us | Duration<br>54.3 us | Duration<br>49.0 us | Duration<br>49.5 us |
| 累计加速<br>1.0×        | 累计加速<br>1.13×       | 累计加速<br>67.7×        | 累计加速<br>91.5×       | 累计加速<br>141.7×      | 累计加速<br>157.0×      | 累计加速<br>—           |





# RmsNorm性能优化

0

Naive 基线

Duration

7693 us

累计加速

1.0×

1

Gamma 预加载

Duration

6790 us

累计加速

1.13×

2

多核并行

Duration

113.6 us

累计加速

67.7×

3

RegBase重构

Duration

84.1 us

累计加速

91.5×

4

Double Buffer

Duration

54.3 us

累计加速

141.7×

5

UB 多行批处理

Duration

49.0 us

累计加速

157.0×

6

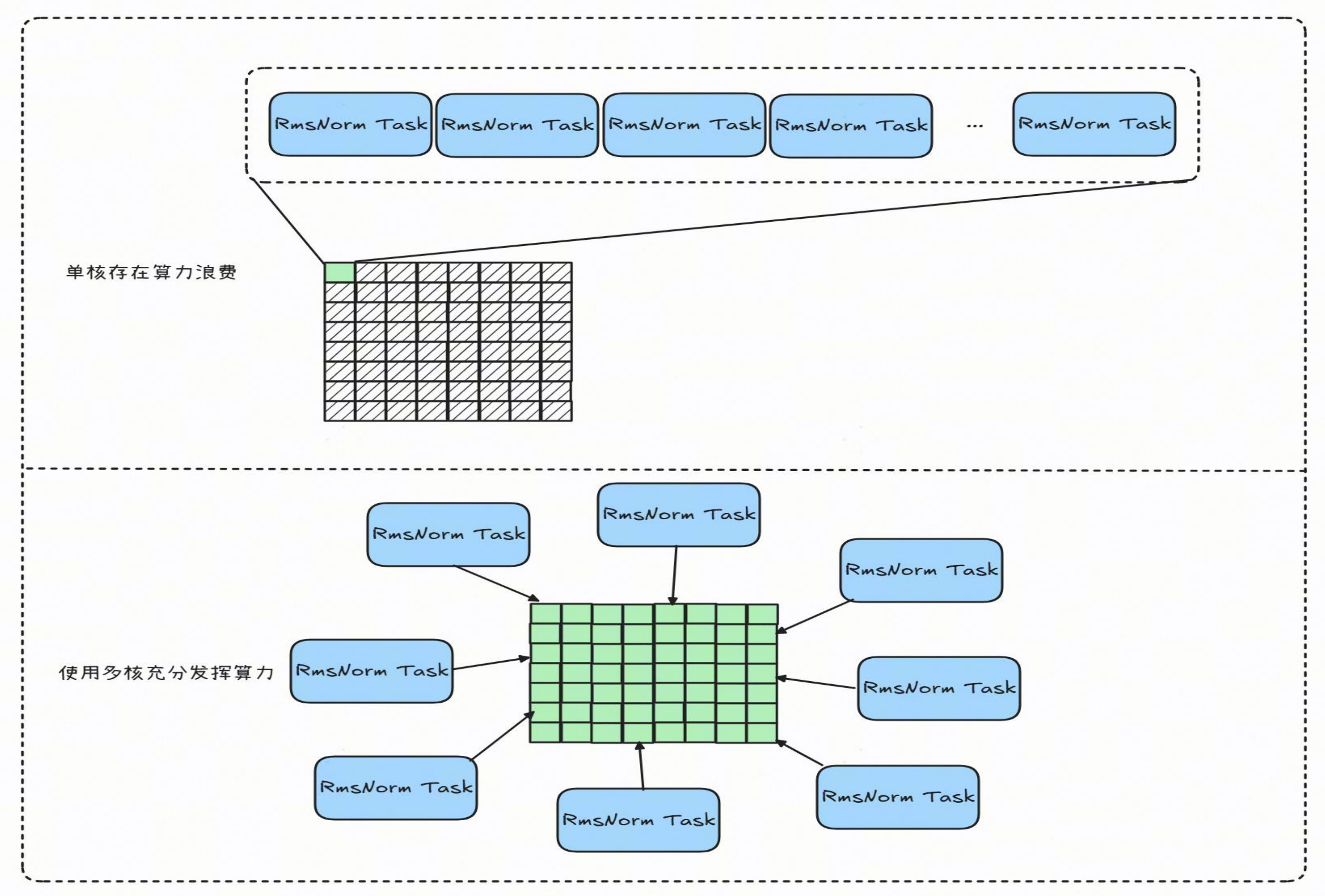
二分累加 + rsqrt

Duration

49.5 us

累计加速

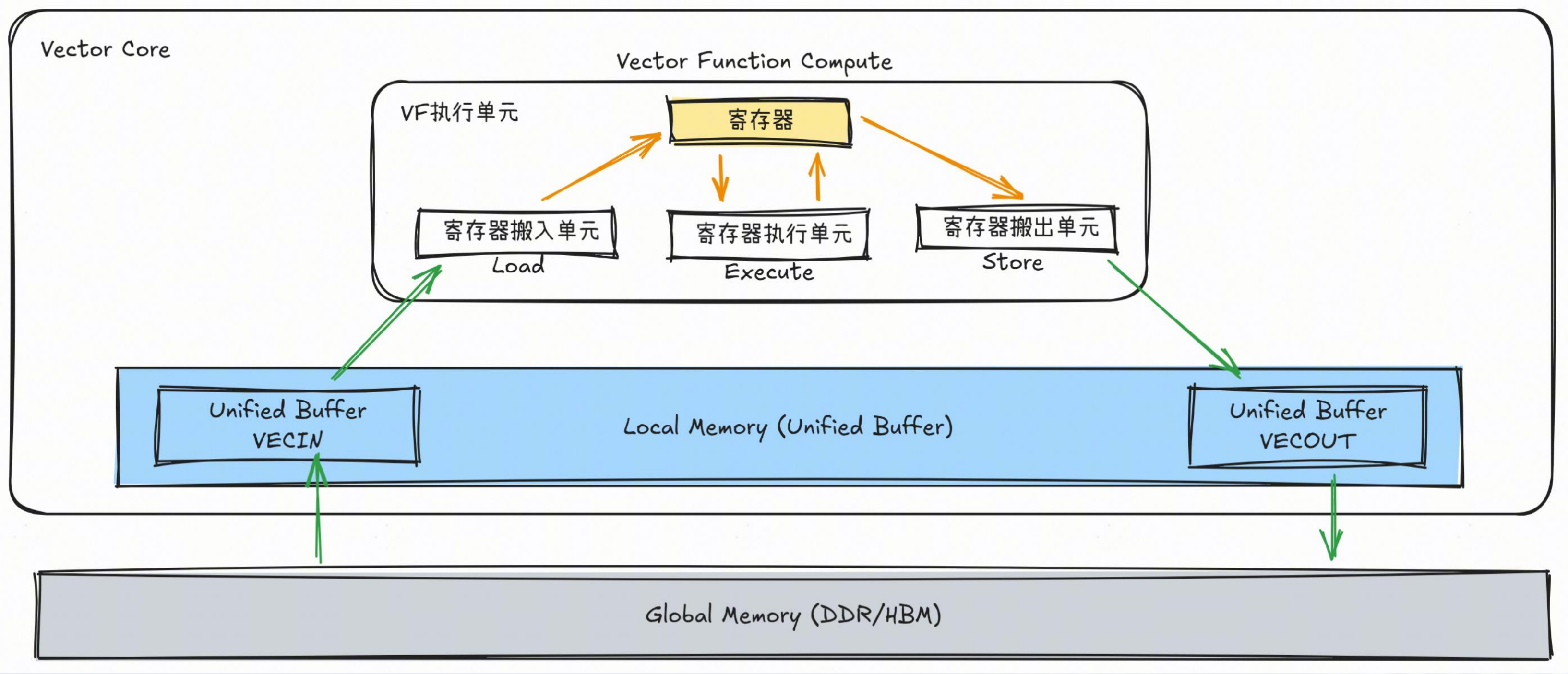
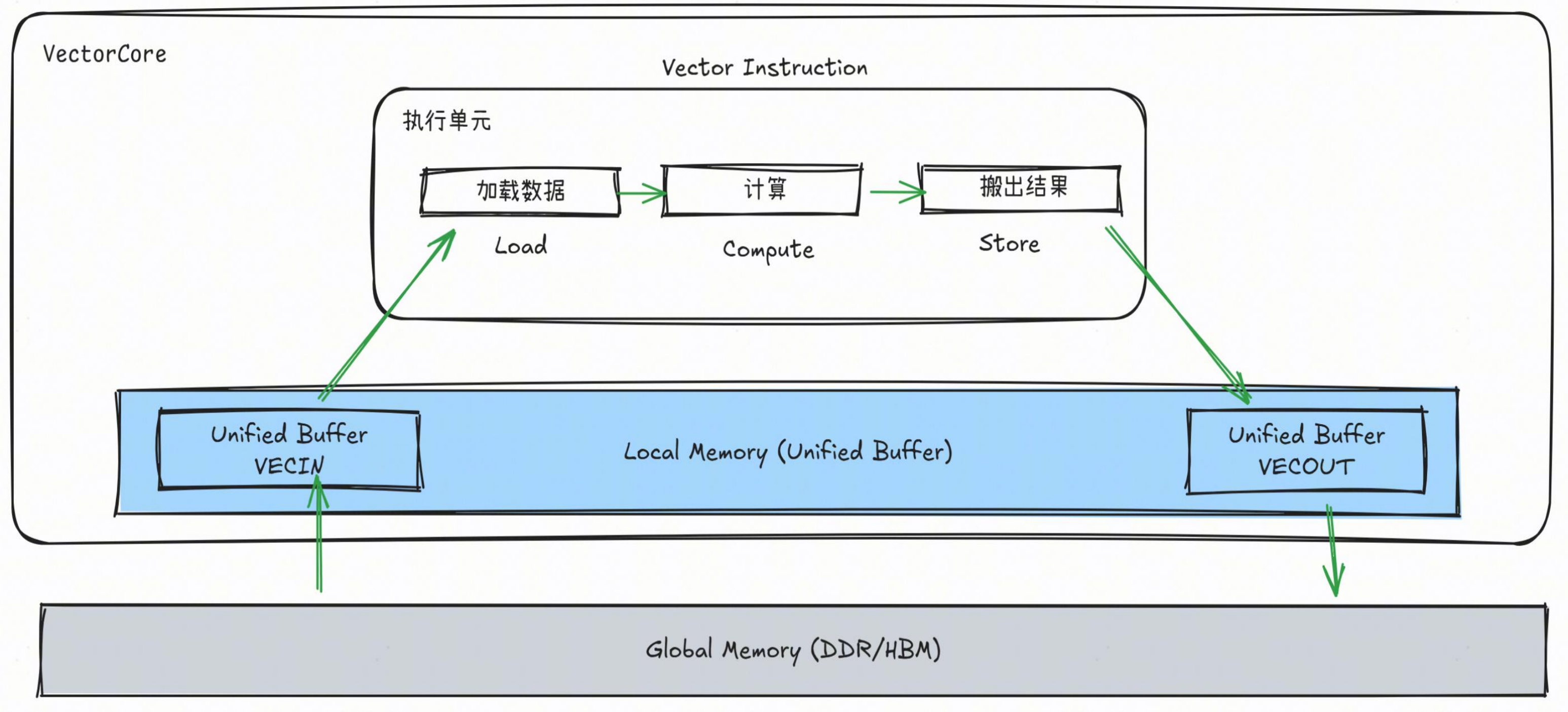
—





# RmsNorm性能优化

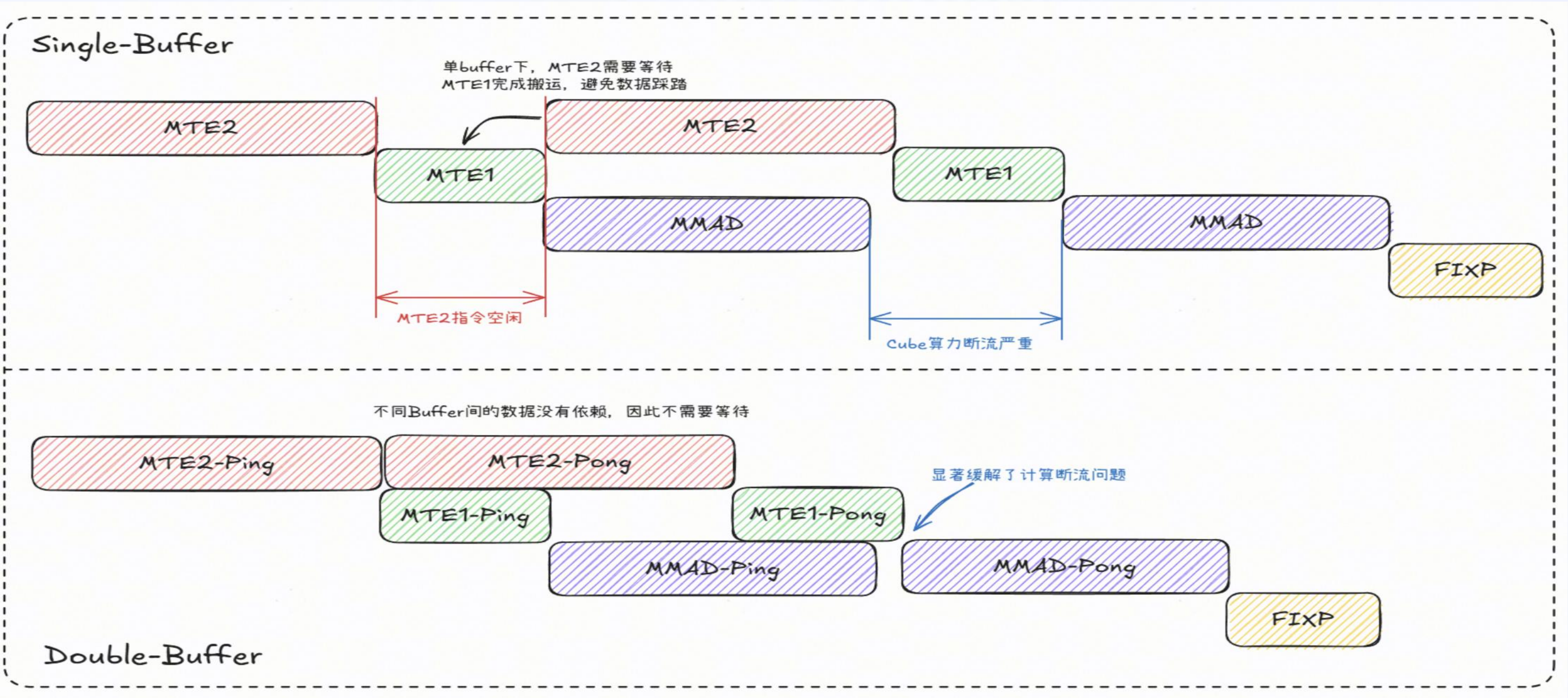
|                                     |                                      |                                       |                                      |                                       |                                       |                                  |
|-------------------------------------|--------------------------------------|---------------------------------------|--------------------------------------|---------------------------------------|---------------------------------------|----------------------------------|
| 0                                   | 1                                    | 2                                     | 3                                    | 4                                     | 5                                     | 6                                |
| Naive 基线                            | Gamma 预加载                            | 多核并行                                  | RegBase重构                            | Double Buffer                         | UB 多行批处理                              | 二分累加 + rsqrt                     |
| Duration<br>7693 us<br>累计加速<br>1.0× | Duration<br>6790 us<br>累计加速<br>1.13× | Duration<br>113.6 us<br>累计加速<br>67.7× | Duration<br>84.1 us<br>累计加速<br>91.5× | Duration<br>54.3 us<br>累计加速<br>141.7× | Duration<br>49.0 us<br>累计加速<br>157.0× | Duration<br>49.5 us<br>累计加速<br>— |





# RmsNorm性能优化

|                     |                     |                      |                     |                     |                     |                     |
|---------------------|---------------------|----------------------|---------------------|---------------------|---------------------|---------------------|
| 0                   | 1                   | 2                    | 3                   | 4                   | 5                   | 6                   |
| Naive 基线            | Gamma 预加载           | 多核并行                 | RegBase重构           | Double Buffer       | UB 多行批处理            | 二分累加 + rsqrt        |
| Duration<br>7693 us | Duration<br>6790 us | Duration<br>113.6 us | Duration<br>84.1 us | Duration<br>54.3 us | Duration<br>49.0 us | Duration<br>49.5 us |
| 累计加速<br>1.0x        | 累计加速<br>1.13x       | 累计加速<br>67.7x        | 累计加速<br>91.5x       | 累计加速<br>141.7x      | 累计加速<br>157.0x      | 累计加速<br>—           |





# RmsNorm性能优化

0

Naive 基线

Duration

7693 us

累计加速

1.0×

1

Gamma 预加载

Duration

6790 us

累计加速

1.13×

2

多核并行

Duration

113.6 us

累计加速

67.7×

3

RegBase重构

Duration

84.1 us

累计加速

91.5×

4

Double Buffer

Duration

54.3 us

累计加速

141.7×

5

UB 多行批处理

Duration

49.0 us

累计加速

157.0×

6

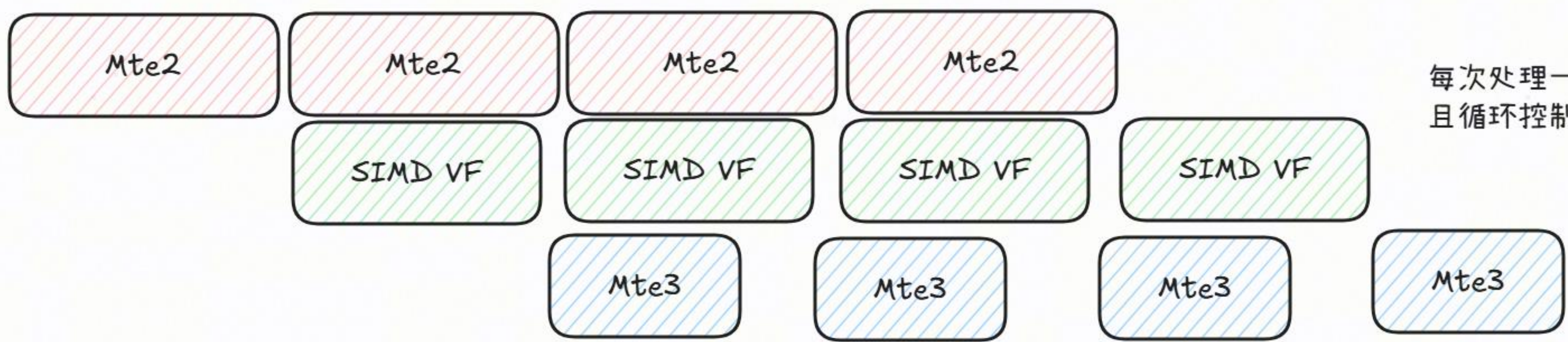
二分累加 + rsqrt

Duration

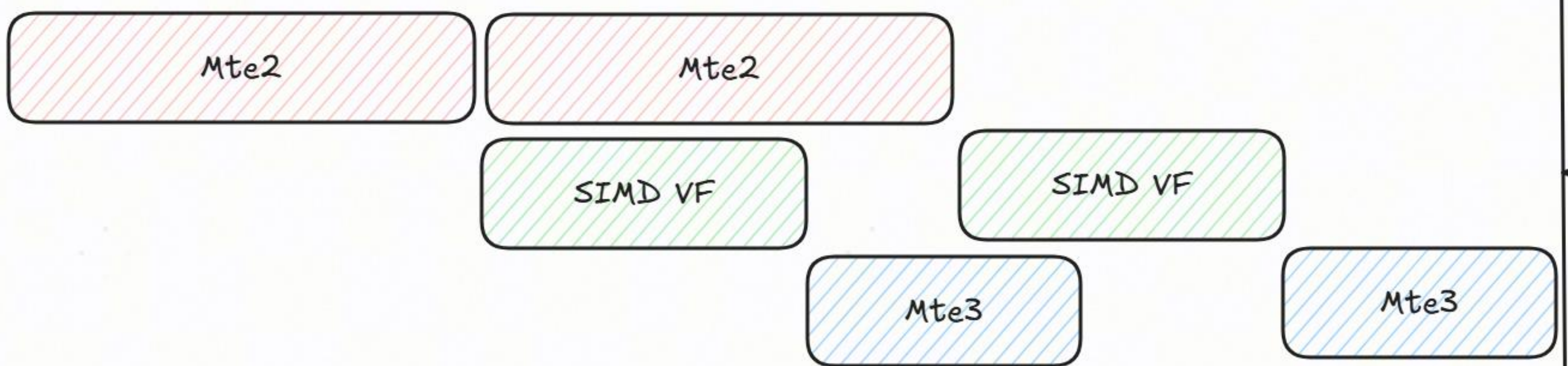
49.5 us

累计加速

—



每次处理一行，UB空间无法充分利用  
且循环控制，指令setup带来额外的开销



一次处理多行摊薄循环控制、  
VF和DMA step的固定成本



# RmsNorm性能优化

0

Naive 基线

Duration

7693 us

累计加速

1.0×

1

Gamma 预加载

Duration

6790 us

累计加速

1.13×

2

多核并行

Duration

113.6 us

累计加速

67.7×

3

RegBase重构

Duration

84.1 us

累计加速

91.5×

4

Double Buffer

Duration

54.3 us

累计加速

141.7×

5

UB 多行批处理

Duration

49.0 us

累计加速

157.0×

6

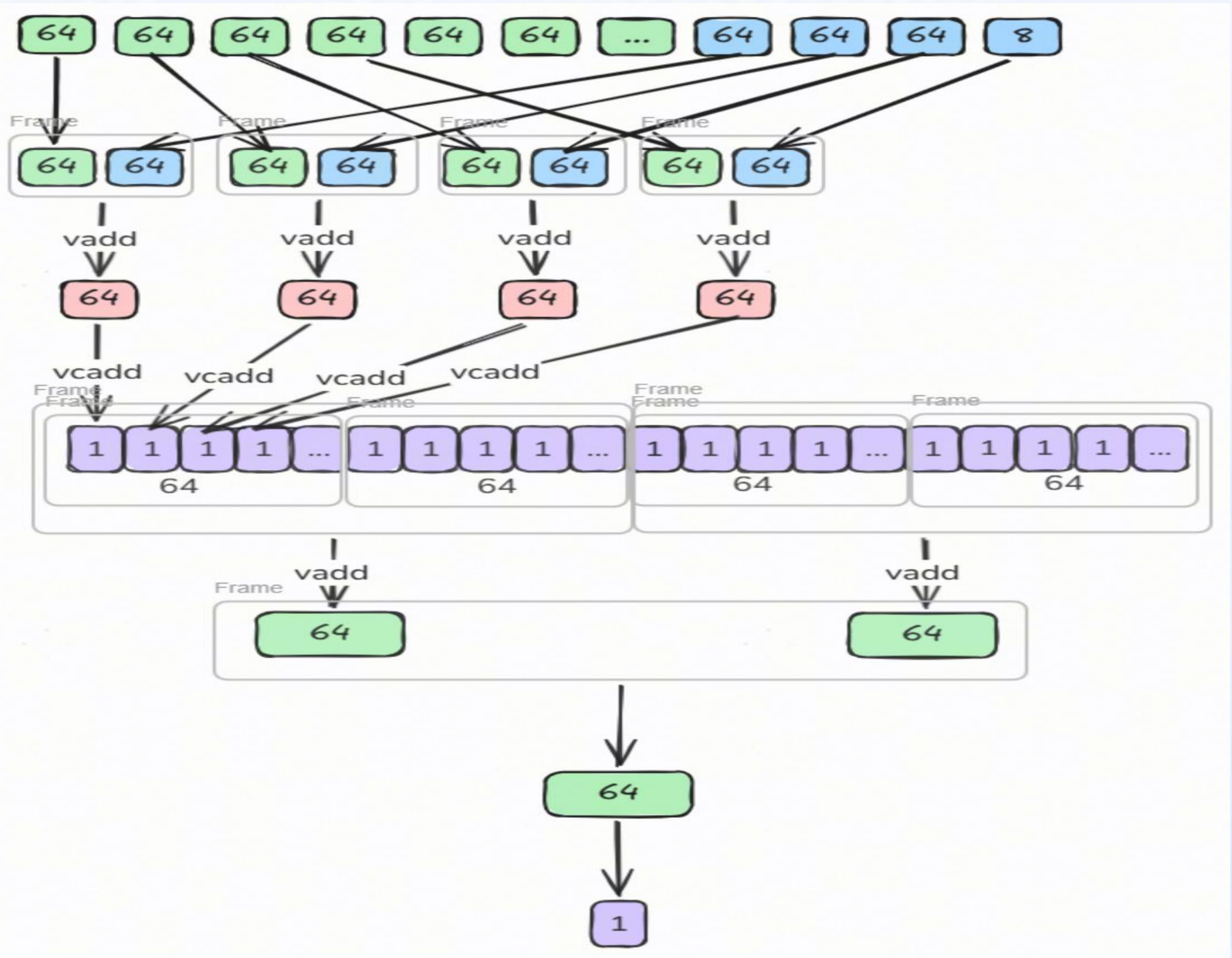
二分累加 + rsqrt

Duration

49.5 us

累计加速

—



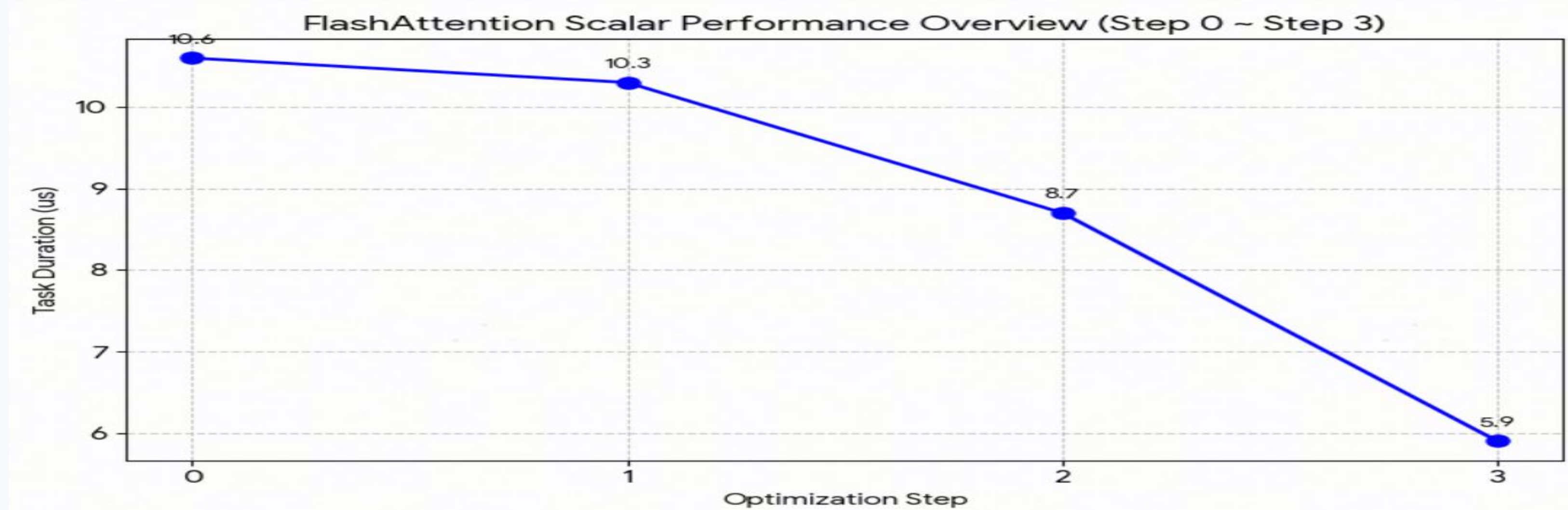


# FlashAttention Scalar性能优化

45%

× 总加速比

10.6 us → 5.9 us



Cube/Mix算子更容易  
ScalarBound

0

Naive 基线

Duration

10.6 us

累计加速

1.0×

1

Lcache 预取

Duration

10.3 us

累计加速

1.03×

2

常量传播

Duration

8.7 us

累计加速

1.22×

3

数据访问局部性增强

Duration

5.9 us

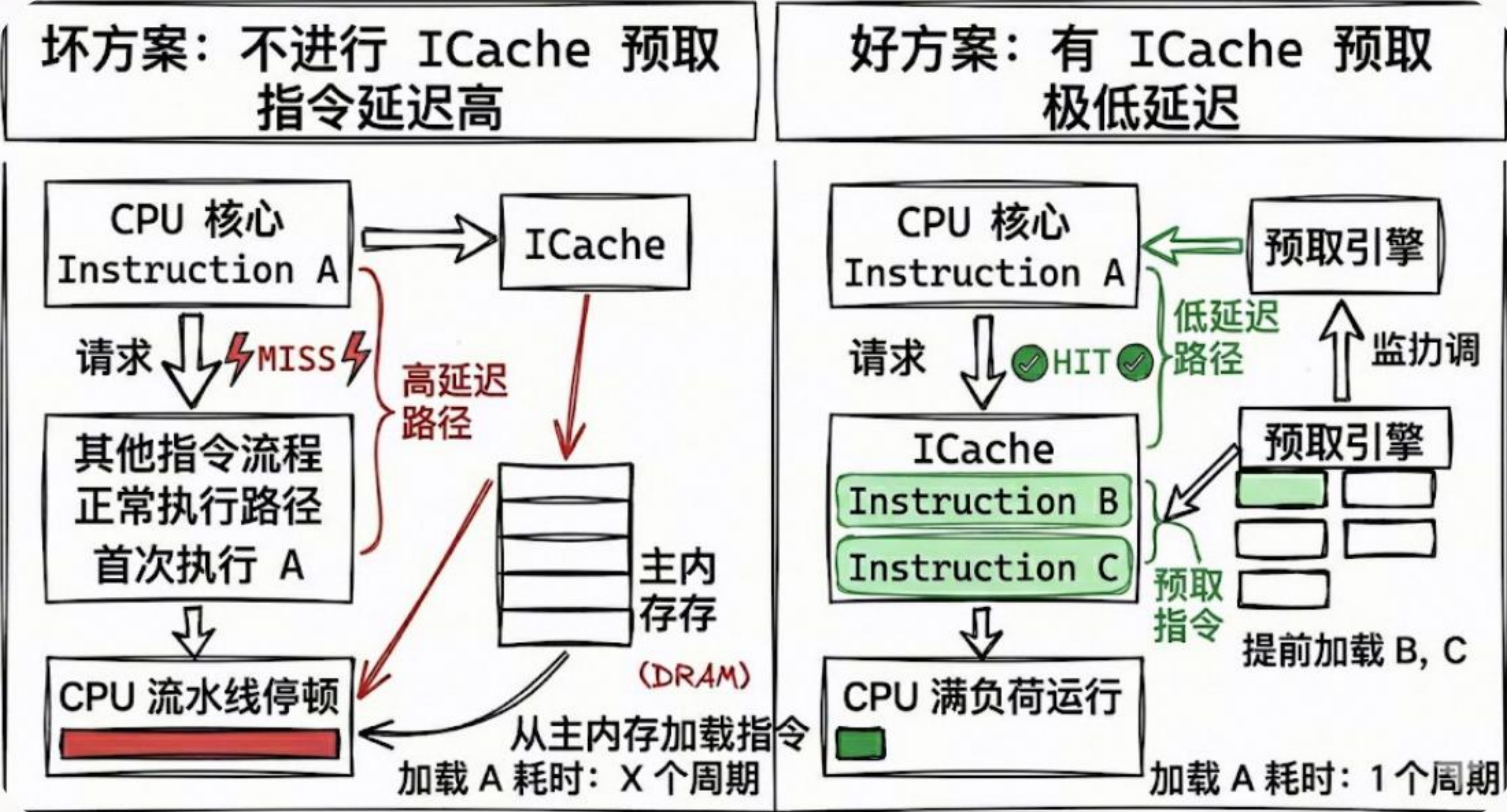
累计加速

1.79×



# FlashAttention Scalar性能优化

|          |           |          |           |
|----------|-----------|----------|-----------|
| 0        | 1         | 2        | 3         |
| Naive 基线 | Icache 预取 | 常量传播     | 数据访问局部性增强 |
| Duration | Duration  | Duration | Duration  |
| 10.6 us  | 10.3 us   | 8.7 us   | 5.9 us    |
| 累计加速     | 累计加速      | 累计加速     | 累计加速      |
| 1.0×     | 1.03×     | 1.22×    | 1.79×     |





# FlashAttention Scalar性能优化

|          |           |          |           |
|----------|-----------|----------|-----------|
| 0        | 1         | 2        | 3         |
| Naive 基线 | Icache 预取 | 常量传播     | 数据访问局部性增强 |
| Duration | Duration  | Duration | Duration  |
| 10.6 us  | 10.3 us   | 8.7 us   | 5.9 us    |
| 累计加速     | 累计加速      | 累计加速     | 累计加速      |
| 1.0×     | 1.03×     | 1.22×    | 1.79×     |

```
for (int32_t i = 0; i < L1KVNumSacler; ++i) {  
    pipe->InitBuffer(tmpBufL1KV[i], KV_L1_SIZE);  
    L1KVTensorPingPong[i] = tmpBufL1KV[i].template Get<T>();  
}
```

LocalTensor动态申请，带来了额外的scalar开销

```
pipe->InitBuffer(tmpBufL1Q, Q_L1_SIZE);  
L1QTensorPingPong = tmpBufL1Q.template Get<T>();  
  
pipe->InitBuffer(tmpBufL1KRope, K_ROPE_SIZE);  
L1KRopeTensor = tmpBufL1KRope.template Get<T>();
```

```
L1KVTensorPingPong0 = LocalTensor<T>(TPosition::B1, (Q_L1_SIZE + 2 * P_L1_SIZE) * 2, KV_L1_SIZE);  
L1KVTensorPingPong1 = LocalTensor<T>(TPosition::B1, (Q_L1_SIZE + 2 * P_L1_SIZE + KV_L1_SIZE) * 2, KV_L1_SIZE);  
L1KVTensorPingPong2 = LocalTensor<T>(TPosition::B1, (Q_L1_SIZE + 2 * P_L1_SIZE + 2 * KV_L1_SIZE) * 2,  
KV_L1_SIZE);  
  
L1QTensorPingPong = LocalTensor<T>(TPosition::A1, 0, Q_L1_SIZE);  
L1KRopeTensor = LocalTensor<T>(TPosition::B1, (Q_L1_SIZE + 2 * P_L1_SIZE + 3 * KV_L1_SIZE) * 2, K_ROPE_SIZE);
```

LocalTensor使用常量静态创建，减少scalar开销



# FlashAttention Scalar性能优化

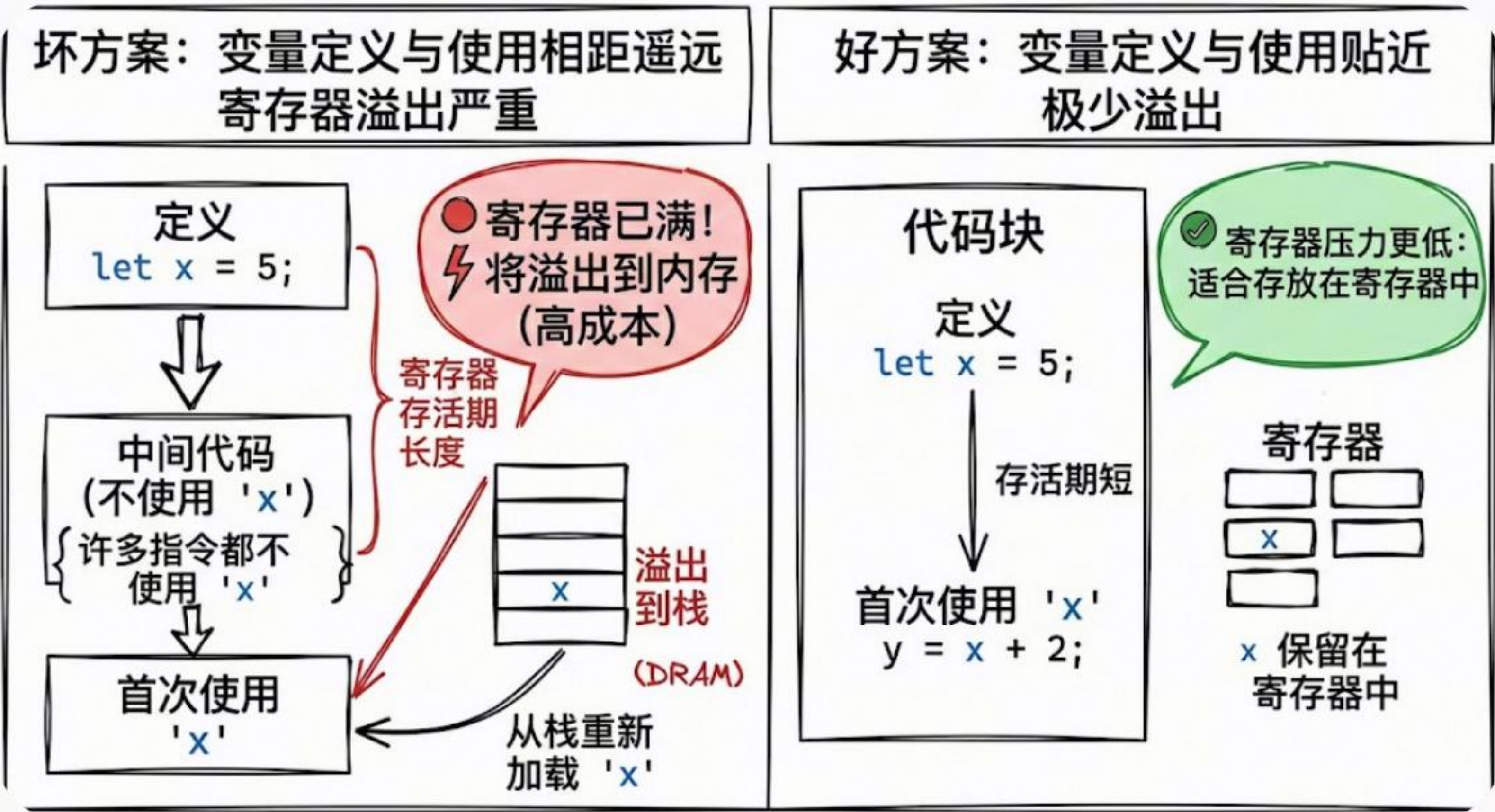
|                     |                     |                    |                    |
|---------------------|---------------------|--------------------|--------------------|
| 0                   | 1                   | 2                  | 3                  |
| Naive 基线            | Lcache 预取           | 常量传播               | 数据访问局部性增强          |
| Duration<br>10.6 us | Duration<br>10.3 us | Duration<br>8.7 us | Duration<br>5.9 us |
| 累计加速<br>1.0x        | 累计加速<br>1.03x       | 累计加速<br>1.22x      | 累计加速<br>1.79x      |

```
template <typename PFAT>
class PromptFlashAttentionNormalBNS1PreloadBasicApi {
    ...
protected:
    ...
    ConstParam constParam;
}

template<typename PFAT>
__aicore__ inline void PromptFlashAttentionNormalBNS1PreloadBasicApi<PFAT>::Init()
{
    ConstParam constParam;
    ...
}
```

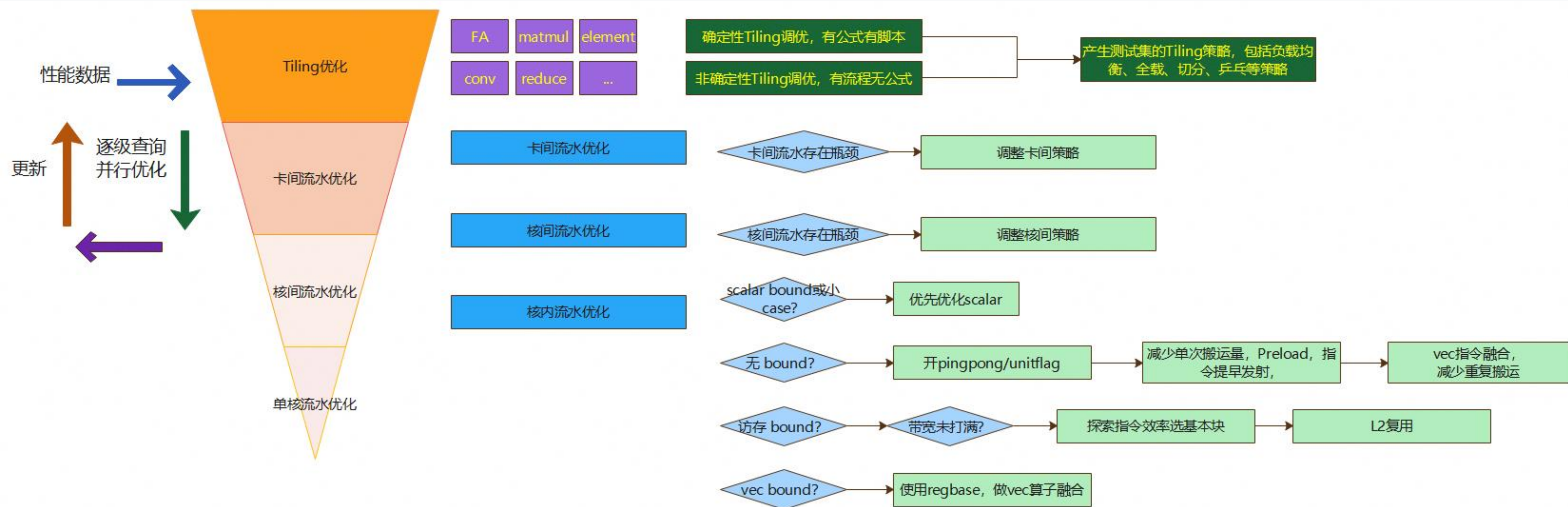
ConstParam为成员变量，生命周期长

constParam对象改为局部变量，且定义贴近使用





# 基于CANNBot的算子自动调优



CANNBot交流群

CANNBot: <https://gitcode.com/cann/cannbot-skills>